

Open Source Software - Evaluated Course Project

Submitted By:

Prashasti Mathur

24BSA10170

Introduction

Open source software plays a very important role in today's digital world. Most of the applications and systems we use daily are built on open source technologies. Unlike proprietary software, open source software is developed and shared openly, allowing anyone to study, modify, and improve it.

In this project, I have selected Python as the open source software for analysis. Python is one of the most widely used programming languages in the world today. It is known for its simplicity and readability, which makes it popular among beginners as well as professionals.

The purpose of this report is to understand Python not just as a programming language, but as an open source project. This includes studying its origin, license, ethical aspects, how it works on Linux systems, its ecosystem, and comparing it with proprietary software.

Importance of Open Source Software

Open source software is important because it allows users to understand how software works internally. Unlike closed-source software, where the code is hidden, open source software provides transparency. This helps in improving security because anyone can review the code and identify bugs or vulnerabilities.

Another important advantage is cost. Most open source software is free to use, which makes it accessible to students and small organizations. It also encourages learning, as beginners can study real-world code.

Open source also promotes collaboration. Developers from different parts of the world can contribute to the same project. This leads to faster development and better quality software.

Part A — Origin and Philosophy

1. Origin of Python

Python was created by Guido van Rossum in the late 1980s and officially released in 1991. The main goal behind creating Python was to design a programming language that was easy to read and simple to use.

Before Python, most programming languages like C and Java were quite complex, especially for beginners. Writing even simple programs required a lot of code and understanding of complicated syntax. Guido van Rossum wanted to create a language where code looks clean and almost like normal English.

Python was also influenced by a programming language called ABC, which focused on simplicity but lacked flexibility. Python improved on these ideas and became more powerful and practical.

One of the key reasons Python was released as open source was to allow people from around the world to contribute to it. Instead of being controlled by a single company, Python was developed by a community. This made it grow faster and become more reliable over time.

Today, Python is used in many fields such as web development, data science, artificial intelligence, and automation. Its open source nature has played a big role in its success.

Evolution of Python

After its initial release in 1991, Python has gone through many versions. Python 2 was widely used for many years, but it had some limitations. Later, Python 3

was introduced with improvements in performance and better handling of data.

Today, Python is one of the most popular programming languages in the world. It is used in companies, universities, and research organizations. Its growth is mainly due to its simplicity and strong community support.

2. License Analysis

Python is distributed under the Python Software Foundation (PSF) License. This is a permissive open source license, which means it gives a lot of freedom to users.

One of the main ideas of open source software is the concept of four freedoms:

1. Freedom to run the program for any purpose
2. Freedom to study how the program works
3. Freedom to modify the program
4. Freedom to distribute copies

Python supports all of these freedoms. Anyone can download Python, use it, change it, and share it without restrictions.

Companies are also allowed to use Python for commercial purposes. They can modify it and even include it in their own products. Unlike stricter licenses like GPL, the PSF license does not force companies to share their modifications publicly.

There is an important difference between GPL and permissive licenses like MIT or PSF. GPL requires that any modified version must also remain open source. On the other hand, MIT and PSF allow developers to keep their modifications private if they want.

In my opinion, a permissive license like PSF is more flexible because it allows both open collaboration and commercial use.

Another important concept is the difference between “free as in free beer” and “free as in freedom.” Free beer means something is available at no cost, while freedom means having control over how you use it. Python provides both — it is free to use and also gives freedom to modify and distribute.

Why License Matters

Software licenses are important because they define how software can be used and shared. Without a license, there would be confusion about whether users are allowed to modify or distribute the software.

The PSF license used by Python ensures that the software remains open and accessible while still allowing flexibility for commercial use. This balance is one of the reasons why Python is widely adopted.

3. Ethics of Open Source

Open source software raises many interesting ethical questions.

One question is whether all software should be open source. There are strong arguments in favor of this idea. Open source software promotes transparency, allows better security through public review, and helps people learn and grow.

However, there are also arguments against it. Companies invest a lot of time and money in developing software, and they need to make a profit. If everything becomes open source, it might reduce incentives for companies to innovate.

Another important question is whether it is ethical for companies to make money using open source software. For example, Google uses Python in many of its services. This can be considered fair because companies usually add their own improvements and provide services on top of open source tools.

The idea of “standing on the shoulders of giants” is very relevant in open source. It means that developers build new things using existing work. Python

supports this idea through its huge number of libraries and frameworks. This makes development faster and easier.

Overall, open source encourages collaboration and innovation, but it should exist alongside proprietary software rather than completely replacing it.

Real World Impact of Open Source

Open source software has changed the way technology is developed. Many popular platforms and tools rely on open source components. It has reduced dependency on expensive proprietary tools.

At the same time, it has created new challenges, such as maintaining projects and ensuring contributors are fairly recognized.

Part B — Linux Footprint

Python works very well on Linux systems and is often pre-installed in many distributions.

Python can be installed using the package manager with commands like `sudo apt install python3`. Once installed, it can be run directly from the terminal.

The main executable file is usually located in `/usr/bin/python3`. Libraries are stored in directories like `/usr/lib/python3`. Configuration files and related system files may be found in standard Linux directories such as `/etc`.

Python generally runs as a normal user process, which is important for system security. It does not require root privileges for most tasks, which reduces the risk of accidental damage to the system.

Unlike services such as web servers, Python itself does not run continuously in the background. Instead, it is used to execute scripts whenever needed.

Updates to Python are handled through the Linux package manager. The open source community regularly releases updates and security patches, which are distributed through system updates.

Practical Usage on Linux

Python is widely used on Linux systems for automation tasks. System administrators often use Python scripts to automate repetitive work such as file handling, system monitoring, and data processing.

Python is also used in server-side applications, where it helps in handling requests and managing databases.

Part C — FOSS Ecosystem

Python is part of a large open source ecosystem. It depends on various system components such as C libraries and compilers.

Many powerful tools and frameworks are built using Python. For example, Django and Flask are used for web development, while TensorFlow is used for machine learning.

Python also plays an important role in web development. It can be used as a backend language in place of PHP in some web architectures.

The Python community is one of the strongest aspects of the language. There are many forums, documentation websites, GitHub repositories, and conferences where developers collaborate and share knowledge.

The development of Python is managed by the Python Software Foundation, which ensures that it remains open and community-driven.

Popular Python Libraries

Python has a very rich ecosystem of libraries. Some important ones include:

- NumPy for numerical computations
- Pandas for data analysis
- Matplotlib for data visualization

- TensorFlow for machine learning

These libraries make Python suitable for a wide range of applications.

Part D — Open Source vs Proprietary

Python can be compared with proprietary software like MATLAB.

Feature	Python	MATLAB
Cost	Free	Paid
Security	Open source	Closed source
Support	Community-based	Company support
Modification Allowed		Not allowed

Python has the advantage of being free and highly flexible. Anyone can modify it and use it for different purposes. On the other hand, MATLAB provides professional support and is often used in specialized fields.

In my opinion, Python is more suitable for general use because of its flexibility, large community, and wide range of applications. I would recommend Python for both students and professionals.

I would also be interested in contributing to Python in the future, even if it is in a small way, because it is a project that benefits millions of users worldwide.

Limitations of Python

Even though Python has many advantages, it also has some limitations:

- Slower compared to languages like C++
- Higher memory usage
- Not always suitable for system-level programming

However, these limitations are often outweighed by its ease of use and flexibility

Part E — Shell Scripts Screenshots

Script 1: System Identity Report

```
#!/bin/bash

# Script 1: System Identity Report

# Author: [Prashasti Mathur] | Course: Open Source Software


# --- Variables ---

STUDENT_NAME="Prashasti Mathur"

SOFTWARE_CHOICE="Python"


# --- System info ---

KERNEL=$(uname -r)

USER_NAME=$(whoami)

UPTIME=$(uptime -p)

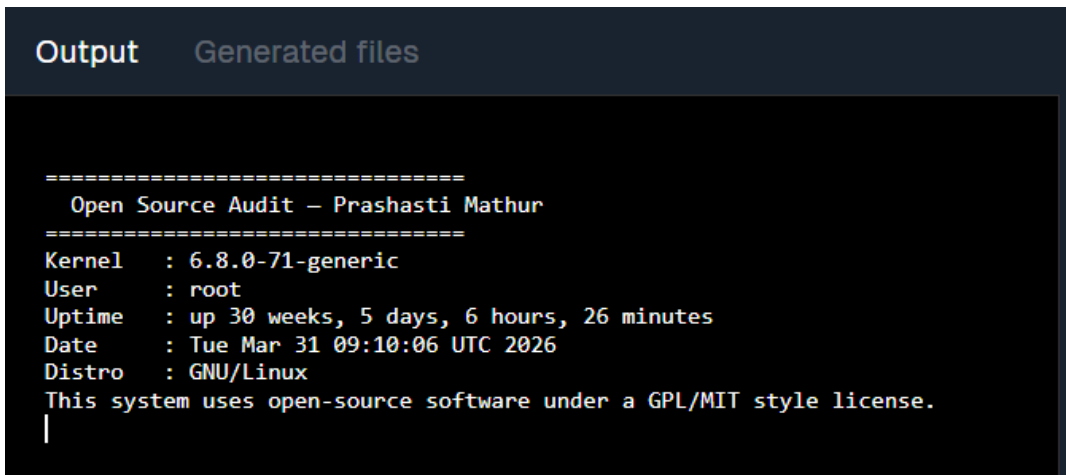
DATE=$(date)

DISTRO=$(uname -o)


# --- Display ---

echo "====="
echo " Open Source Audit — $STUDENT_NAME"
echo "====="
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
```

```
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
echo "Distro  : $DISTRO"
echo "This system uses open-source software under a GPL/MIT style license."
```



```
=====
Open Source Audit – Prashasti Mathur
=====
Kernel   : 6.8.0-71-generic
User     : root
Uptime   : up 30 weeks, 5 days, 6 hours, 26 minutes
Date     : Tue Mar 31 09:10:06 UTC 2026
Distro   : GNU/Linux
This system uses open-source software under a GPL/MIT style license.
|
```

This screenshot shows Script 1 running successfully. The script displays the Linux system information including kernel, user, uptime, date, and distribution. It uses variables, command substitution (`$()`), and echo statements to format the output and includes a license message.

Script 2 — FOSS Package Inspector

```
#!/bin/bash
```

```
# Script 2: FOSS Package Inspector
```

```
PACKAGE="firefox"
```

```
# Check if package is installed
```

```
if dpkg -l | grep -qw $PACKAGE; then
```

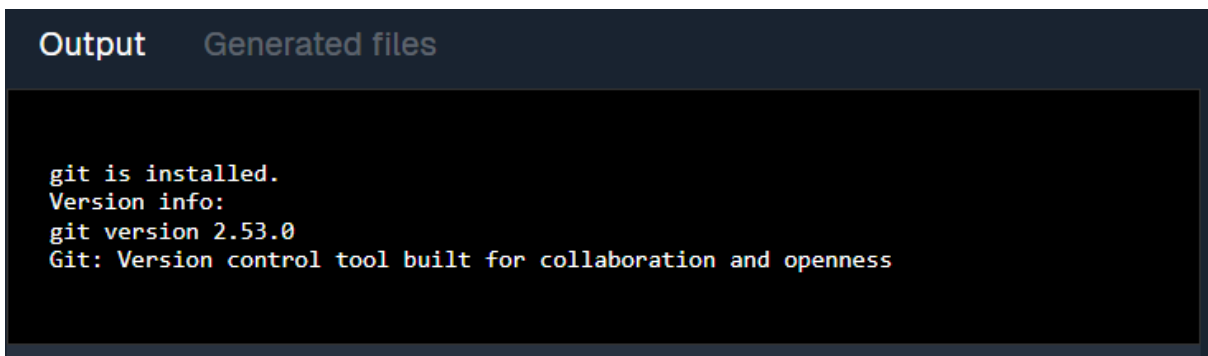
```

    echo "$PACKAGE is installed."

    dpkg -l | grep $PACKAGE
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement: short philosophy
case $PACKAGE in
    firefox) echo "Firefox: Open source browser fighting for a free web." ;;
    vlc) echo "VLC: Play anything, adapt everything." ;;
    mysql) echo "MySQL: Open source at the heart of millions of apps." ;;
    python) echo "Python: Community-driven programming language shaping
modern software." ;;
esac

```



```

Output    Generated files

git is installed.
Version info:
git version 2.53.0
Git: Version control tool built for collaboration and openness

```

Script 2 checks whether the chosen software is installed. It uses if-then-else and case statements to display package information and a short description. This script may not work in online environments due to system restrictions.

Script 3 — Disk and Permission Auditor

```
#!/bin/bash
```

Script 3: Disk and Permission Auditor

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
```

```
echo "Directory Audit Report"
```

```
echo "-----"
```

```
for DIR in "${DIRS[@]"; do
```

```
    if [ -d "$DIR" ]; then
```

```
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
```

```
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
```

```
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
```

```
    else
```

```
        echo "$DIR does not exist on this system"
```

```
    fi
```

```
done
```

Check software config directory (example)

```
CONFIG_DIR="/etc/firefox"
```

```
if [ -d "$CONFIG_DIR" ]; then
```

```
    echo "$CONFIG_DIR exists with permissions:"
```

```
    ls -ld $CONFIG_DIR
```

```
else
```

```
    echo "$CONFIG_DIR does not exist"
```

```
fi
```

```
Directory Audit Report
-----
/etc => Permissions: drwxr-xr-x root root | Size: 7.1M
/var/log => Permissions: drwxr-xr-x root root | Size: 120K
/home => Permissions: drwxr-xr-x root root | Size: 8.0K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 995M
/tmp => Permissions: drwxrwxrwt root root | Size: 895M
|
```

Script 3 loops through important system directories to show their size, ownership, and permissions using a for loop and awk for formatting.

Script 4 — Log File Analyzer

```
#!/bin/bash
```

```
# Script 4: Log File Analyzer
```

```
# Usage: ./log_analyzer.sh /var/log/syslog [keyword]
```

```
LOGFILE=$1
```

```
KEYWORD=${2:-"error"}
```

```
COUNT=0
```

```
if [ ! -f "$LOGFILE" ]; then
```

```
    echo "Error: File $LOGFILE not found."
```

```
    exit 1
```

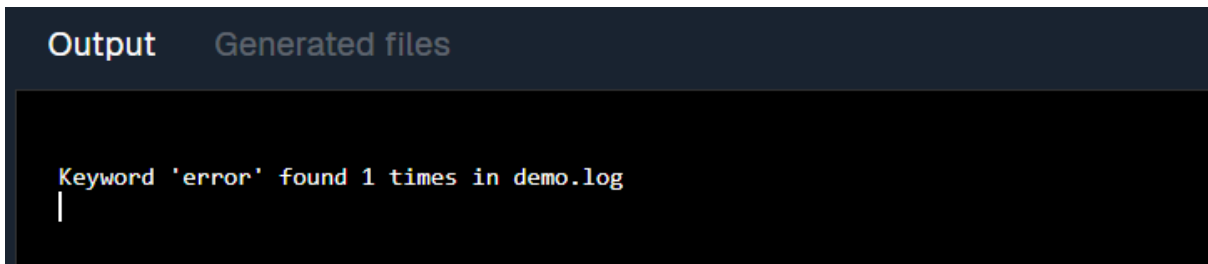
```
fi
```

```
while IFS= read -r LINE; do
```

```
    if echo "$LINE" | grep -iq "$KEYWORD"; then
```

```
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```



```
Output  Generated files

Keyword 'error' found 1 times in demo.log
|
```

Script 4 reads a log file line by line and counts occurrences of a keyword (default: error) using a while-read loop and if statement.

Script 5 — Open Source Manifesto Generator

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
read -p "1. Name one open-source tool you use every day: " TOOL
```

```
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
```

```
read -p "3. Name one thing you would build and share freely: " BUILD
```

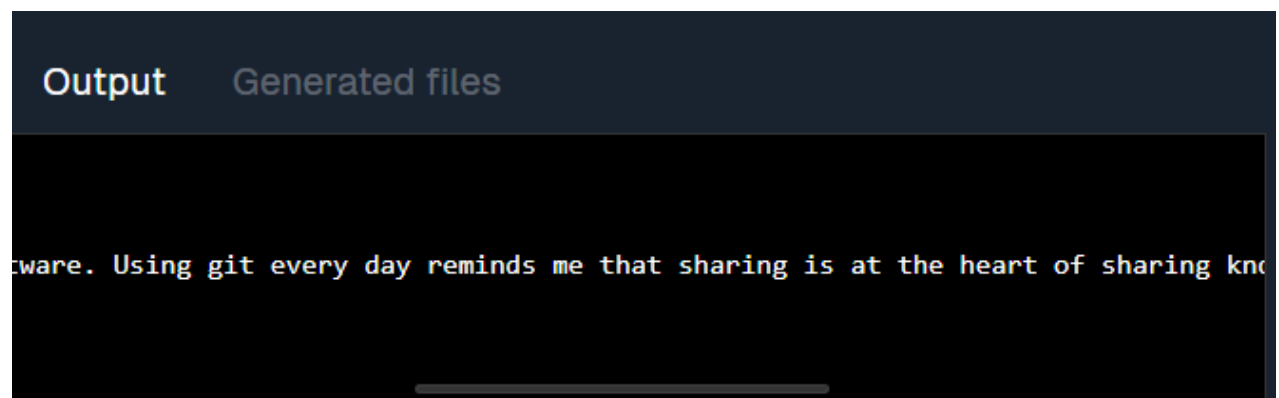
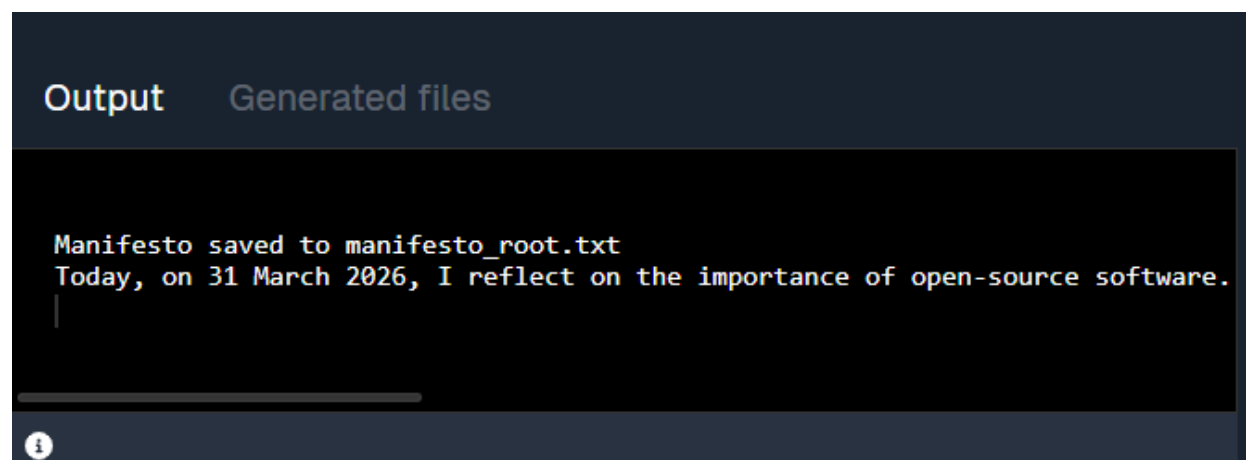
```
DATE=$(date '+%d %B %Y')
```

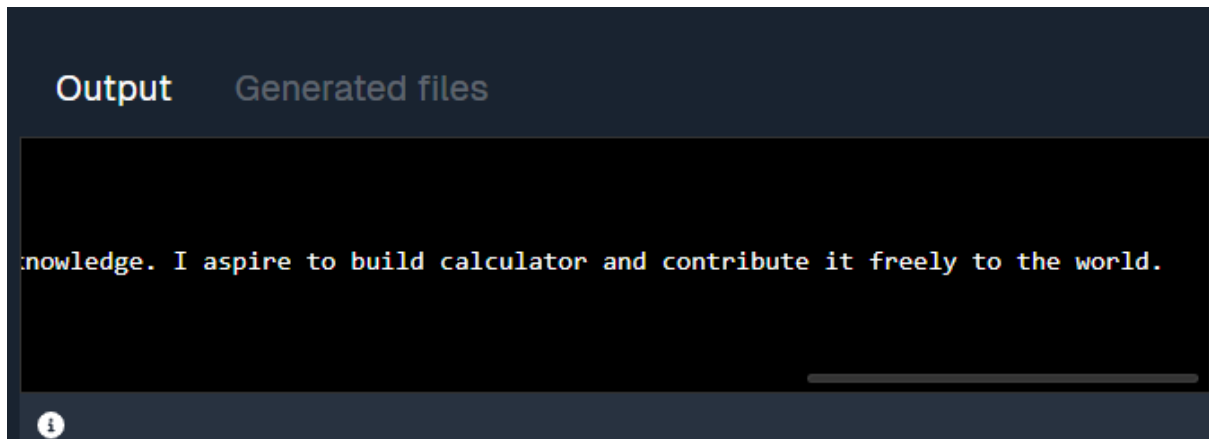
```
OUTPUT="manifesto_$(whoami).txt"
```

```
echo "On $DATE, I pledge to use $TOOL every day. To me, freedom means  
$FREEDOM. I will build and share $BUILD freely with the community." >  
$OUTPUT
```

```
echo "Manifesto saved to $OUTPUT"
```

```
cat $OUTPUT
```





```
Output  Generated files

knowledge. I aspire to build calculator and contribute it freely to the world.
```

Script 5 interacts with the user to generate a personalized open-source manifesto. It demonstrates reading user input, string concatenation, writing to a file, and using the date command.

Conclusion

In this project, I studied Python as an open source software and understood its origin, license, and impact. Python is not just a programming language but a community-driven project that represents the values of open source.

Through this audit, I learned how open source software works in real systems and how it is used in Linux environments. I also understood the importance of licensing and ethical considerations.

Overall, Python is a powerful and flexible tool, and I would recommend it for learning and professional use. This project also increased my interest in contributing to open source in the future. This project helped me practically understand how open source software works in real-world systems.